

SmoIVLM

Redefining Small and Efficient Multimodal Models

Hugging Face & Stanford University

Large VLMs Are Powerful but Impractical for Edge Deployment

State-of-the-Art Reality:

Current flagship VLMs require massive compute and memory, preventing local execution on mobile and edge devices.

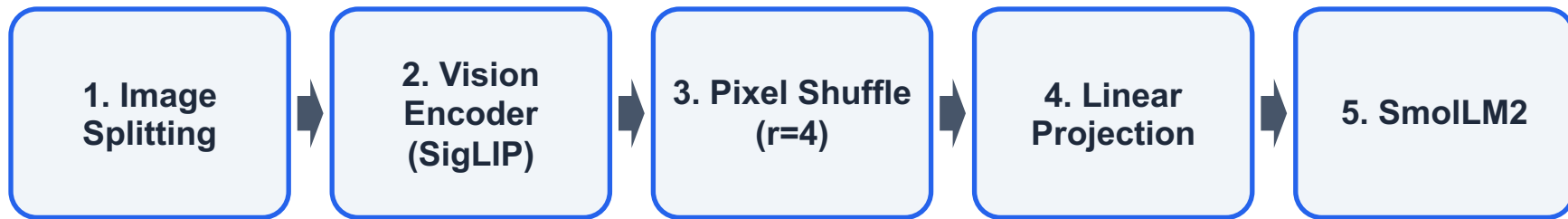
Memory Bottlenecks:

- Molmo-A1B-7B requires 27.7 GB of GPU RAM to run.
- Llama-3.1-8B uses 80 billion parameters, requiring multi-GPU setups.

The Goal:

A compact VLM architecture enabling sub-1GB local inference, preserving privacy, low latency, and offline availability.

SmolVLM Architecture: Image Splitting & Pixel Shuffle

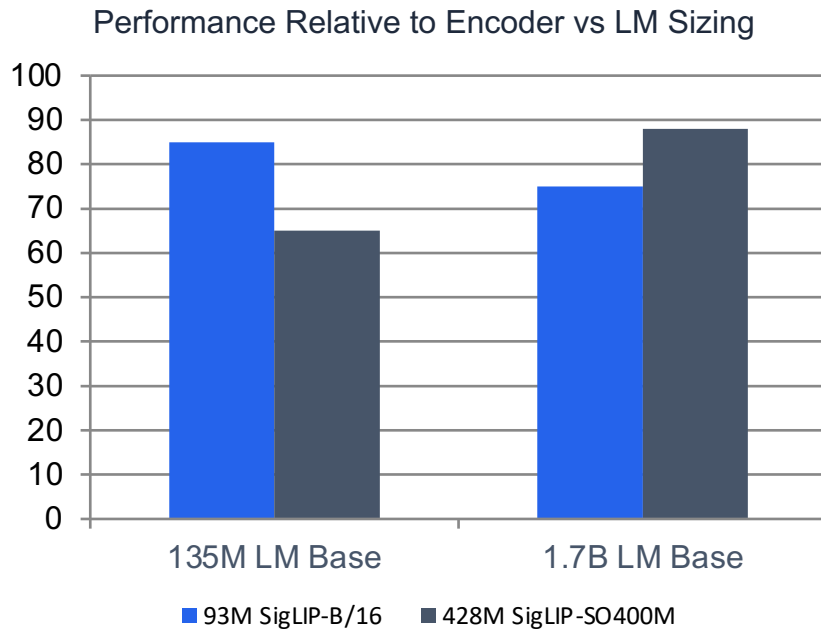


Pipeline Flow: High-resolution images are split into sub-images, encoded by a lightweight SigLIP vision encoder, compressed heavily via space-to-depth pixel shuffle, projected linearly, and finally fed into the SmolLM2 language model token space.

Finding 1: Smaller Vision Encoders Outperform

Balanced Compute Allocation

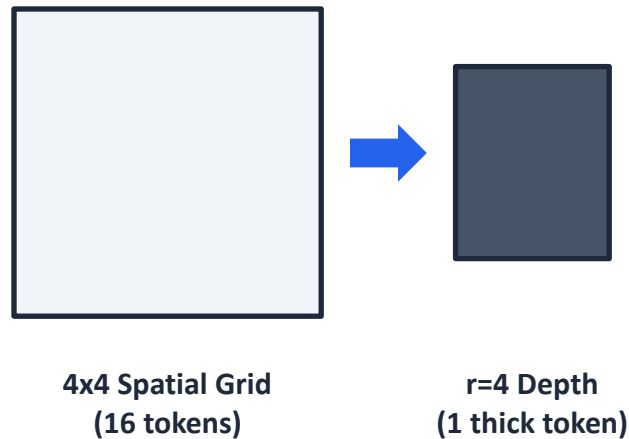
A critical discovery for compact VLMs is that matching a small language model with an oversized vision encoder degrades performance. The vision encoder must be scaled relative to the LLM backbone.



Finding 3: Aggressive Pixel Shuffle ($r=4$) Compression

16x Visual Token Reduction

- Pixel shuffle (space-to-depth) rearranges spatial features into depth channels.
- While large VLMs typically use $r=2$ (4x reduction), small VLMs strongly benefit from $r=4$ (16x reduction).
- Eases attention overhead, enabling high-resolution and multi-image modeling.



Finding 2: Extending Context Length (2K to 16K)

Scaling Context Unlocks High Resolution

Extending context length is critical for multiple sub-images and video sequences.

Methodology:

- Increased the RoPE (Rotary Position Embedding) base from 10K to 273K.
- Fine-tuned on a heavily curated mixture of long and short-context visual data.

Result: The SmolVLM 2.2B model supports full 16K context; smaller 256M/500M variants effectively utilize an 8K context window.

SmolVLM Achieves Competitive Performance

Capability	SmolVLM 256M	SmolVLM 500M	SmolVLM 2.2B	Best Efficient OS
Single-Image	55.3%	64.7%	74.8%	70.1% (Molmo-A1B-7B)
Multi-task	33.2%	37.4%	46.5%	38.2% (Molmo-A1B-7B)
Video	37.2%	44.8%	53.7%	46.6% (InternVL2-2B)
Average All	44.0%	51.0%	59.8%	-
GPU RAM Usage	0.8 GB	1.2 GB	4.9 GB	27.7 GB (Molmo)

Highlight: SmolVLM-256M surpasses the 300x larger Llafix-80B, proving extreme architectural efficiency.

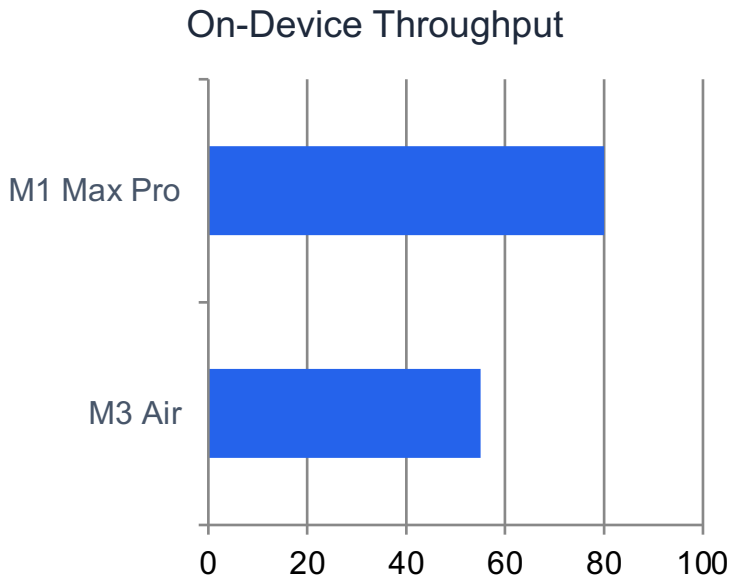
Real-Time Inference via HuggingSnap on Edge

Zero Cloud Dependency

Deployed via the HuggingSnap iOS app, ensuring absolute privacy, zero network latency, and offline availability.

Throughput (SmolVLM-256M):

- Apple Pro M1 Max (64GB): ~80 tokens/sec
- Apple MacBook Air M3 (16GB): ~55 tokens/sec
- **Batch Size 64 Optimization: Scales to nearly 1,000 tokens/sec**



Key Takeaways: Design Principles for Efficient VLMs

1. Balance Encoder-LM Compute

Smaller LMs require downscaled vision encoders (e.g., 93M SigLIP) to maximize efficiency.

2. Extended Context is Mandatory

Scaling from 2K to 16K unlocks complex video and high-res image capabilities.

3. Aggressive Token Compression Works

Applying $r=4$ pixel shuffle (16x reduction) drastically reduces multi-image attention overhead.

4. Sub-1GB Inference Changes the Paradigm

SmolVLM-256M proves deliberate architecture outperforms raw parameter scale on